T08

EXCEPCIONES

INTRODUCCIÓN

- Un programa Python finaliza tan pronto como encuentra un error.
- En Python, un error puede ser un error de sintaxis o una excepción.
- Veremos qué es una excepción y en qué se diferencia de un error de sintaxis.
- Aprenderemos a lanzar excepciones y hacer aserciones.
- Crearemos nuestras propias excepciones

ERROR DE SINTAXIS

```
>>> print( 0 / 0 ))
  File "<stdin>", line 1
    print( 0 / 0 ))
                  ^
SyntaxError: invalid syntax
```

ERROR DE EXCEPCIÓN

```
>>> print(0 / 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: integer division or modulo by zero
```

Este tipo de error ocurre siempre que el código Python sintácticamente correcto genera un error.

GENERAR UNA EXCEPCIÓN

Use raise to force an exception:

```
x = 10
if x > 5:
    raise Exception('x should not exceed 5. The value
of x was: {}'.format(x))
```

```
Traceback (most recent call last):
  File "<input>", line 4, in <module>
Exception: x should not exceed 5. The value of x
was: 10
```

LA EXCEPCIÓN ASSERTIONERROR

- En lugar de esperar a que un programa falle a mitad de camino, podemos comenzar haciendo una aserción (*assert*) en Python.
- Afirmamos que se cumple una determinada condición.
  - Si esta condición resulta ser verdadera, el programa puede continuar.
  - Si la condición resulta ser falsa, podemos hacer que el programa genere una excepción *AssertionError*.

LA EXCEPCIÓN ASSERTIONERROR

Assert that a condition is met:

assert:

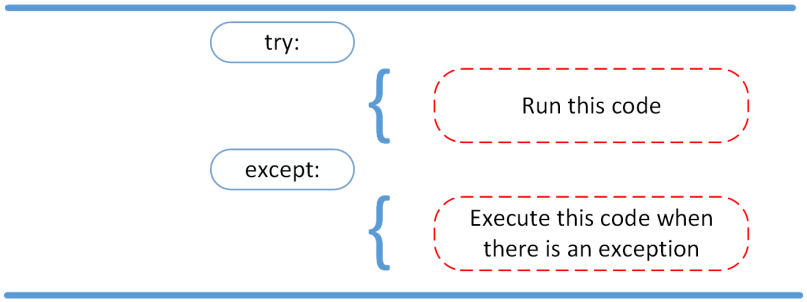
Test if condition is True

```
import sys
assert ('linux' in sys.platform), "This code runs on Linux only."


Traceback (most recent call last):
  File "<input>", line 2, in <module>
AssertionError: This code runs on Linux only.
```

Lo último que hará el programa es Lanzar una excepción *AssertionError*. El programa se detendrá y no continuará. ¿Qué pasa si eso no es lo que queremos?

BLOQUES TRY Y EXCEPT

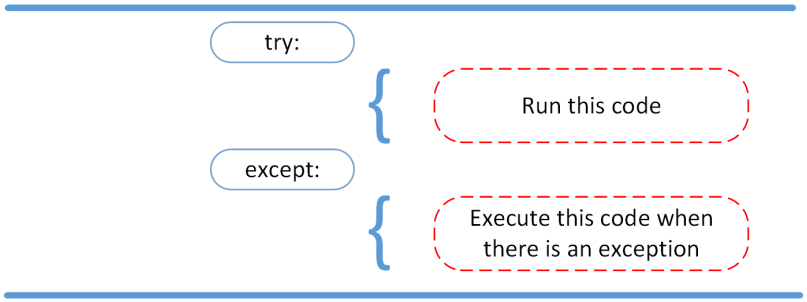



```
def linux_interaction():
    assert ('linux' in sys.platform), \
             "Function can only run on Linux systems."
    print('Doing something.')


try:
    linux_interaction()
except:
    pass
```

¿Terminará el programa? ¿Cuál será la salida?

BLOQUES TRY Y EXCEPT

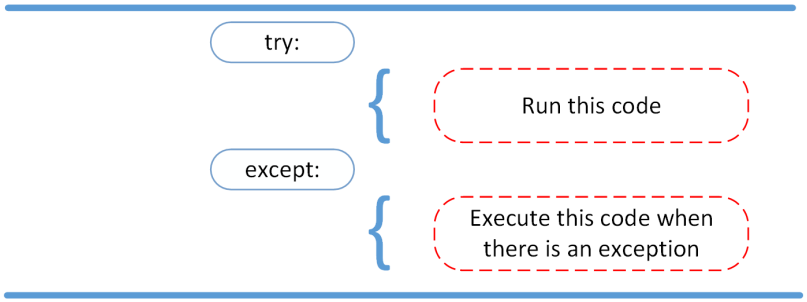



```
def linux_interaction():
    assert ('linux' in sys.platform), \
             "Function can only run on Linux systems."
    print('Doing something.')


try:
    linux_interaction()
except:
    print('Linux function was not executed')
```

¿Terminará el programa? ¿Cuál será la salida?

BLOQUES TRY Y EXCEPT

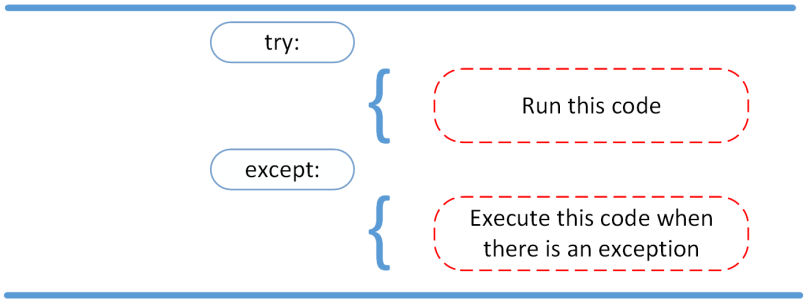



```
def linux_interaction():
    assert ('linux' in sys.platform), \
             "Function can only run on Linux systems."
    print('Doing something.')


try:
    linux_interaction()
except AssertionError as error:
    print(error)
    print('The linux_interaction() function was not executed')
```

¿Terminará el programa? ¿Cuál será la salida?

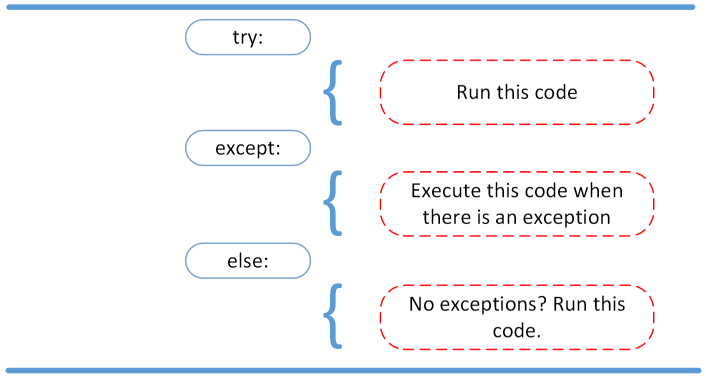
BLOQUES TRY Y EXCEPT




```
try:
    linux_interaction()
    with open('file.log') as file:
        read_data = file.read()
except FileNotFoundError as fnf_error:
    print(fnf_error)
except AssertionError as error:
    print(error)
    print('Linux linux_interaction() function was not executed')
```

Debemos evitar usar *except* de forma genérica. Siembre debe ir acompañado del tipo de excepción que queremos capturar

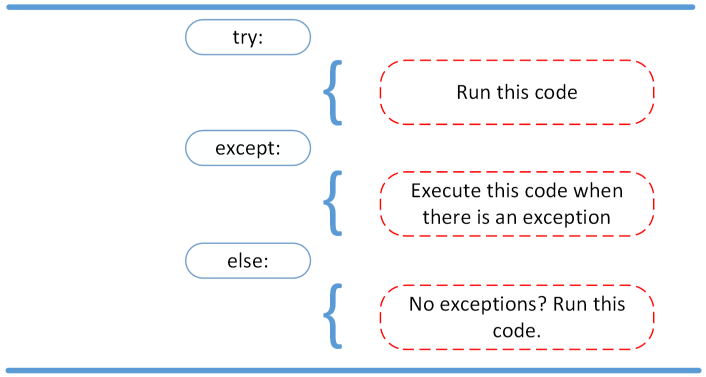
BLOQUE ELSE




```
try:
    linux_interaction()
except AssertionError as error:
    print(error)
else:
    print('Executing the else clause.')
```

¿Terminará el programa? ¿Cuál será la salida?

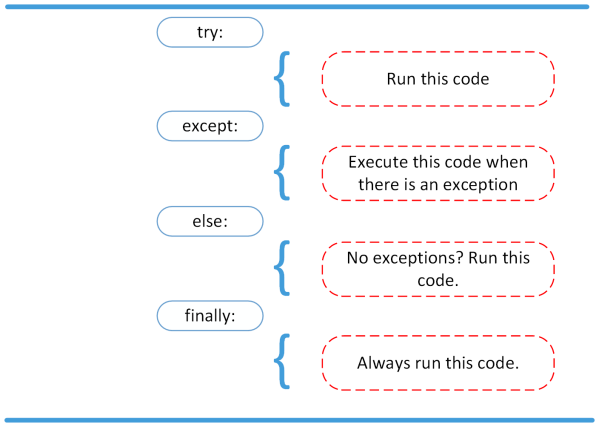
BLOQUE ELSE




```
try:
    linux_interaction()
except AssertionError as error:
    print(error)
else:
    try:
        with open('file.log') as file:
            read_data = file.read()
    except FileNotFoundError as fnf_error:
        print(fnf_error)
```

¿Terminará el programa? ¿Cuál será la salida?

BLOQUE FINALLY




```
try:
    linux_interaction()
except AssertionError as error:
    print(error)
else:
    try:
        with open('file.log') as file:
            read_data = file.read()
    except FileNotFoundError as fnf_error:
        print(fnf_error)
finally:
    print('Cleaning up, irrespective of any exceptions.')
```

¿Terminará el programa? ¿Cuál será la salida?

RESUMEN

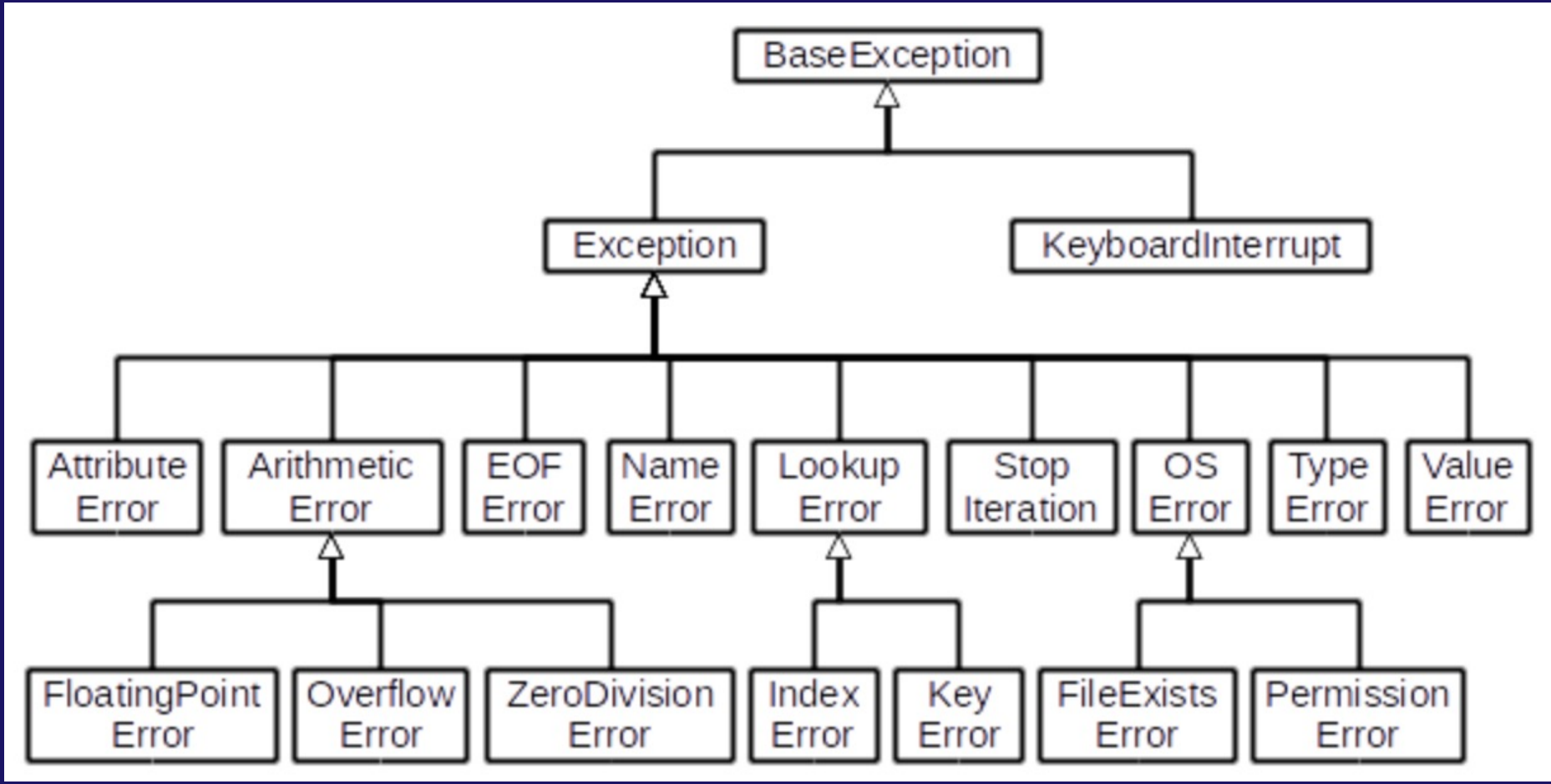
- raise: lanzar una excepción en cualquier momento.
- assert: verificar si se cumple una determinada condición y generar una excepción si no es así.
- try: todo el código se ejecuta hasta que se encuentra una excepción.
- except: capturar y manejar las excepciones que se encuentran en la cláusula try.
- else: ejecutarse solo cuando no se encuentren excepciones en la cláusula try.
- finally: siempre debería ejecutarse, con o sin excepciones encontradas previamente.

EJERCICIO 8.1

EXCEPCIONES ESTANDAR EN PYTHON

Exception	Description
ArithmeticError	Raised when an error occurs in numeric calculations
AssertionError	Raised when an assert statement fails
AttributeError	Raised when attribute reference or assignment fails
Exception	Base class for all exceptions
EOFError	Raised when the input() method hits an "end of file" condition (EOF)
FloatingPointError	Raised when a floating point calculation fails
ImportError	Raised when an imported module does not exist
IndexError	Raised when an index of a sequence does not exist
KeyError	Raised when a key does not exist in a dictionary
KeyboardInterrupt	Raised when the user presses Ctrl+c, Ctrl+z or Delete
MemoryError	Raised when a program runs out of memory
NameError	Raised when a variable does not exist
NotImplementedError	Raised when an abstract method requires an inherited class to override the method
OSError	Raised when a system related operation causes an error
OverflowError	Raised when the result of a numeric calculation is too large
ReferenceError	Raised when a weak reference object does not exist
RuntimeError	Raised when an error occurs that do not belong to any specific exceptions
SyntaxError	Raised when a syntax error occurs
SystemError	Raised when a system error occurs
SystemExit	Raised when the sys.exit() function is called
TypeError	Raised when two different types are combined
ValueError	Raised when there is a wrong value in a specified data type
ZeroDivisionError	Raised when the second operator in a division is zero

EXCEPCIONES ESTANDAR EN PYTHON

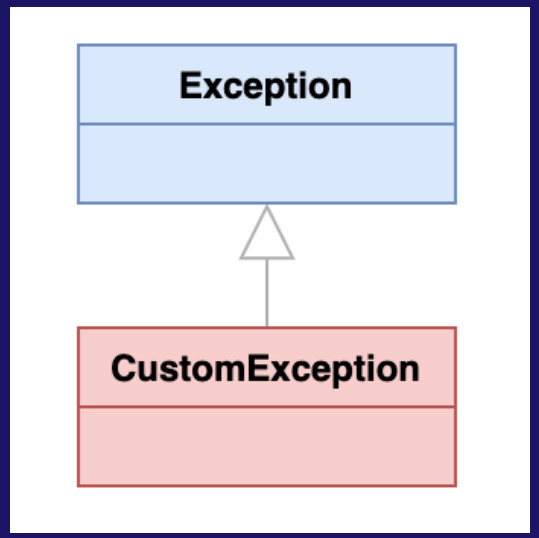


EJERCICIO 8.2

EXCEPCIONES PERSONALIZADAS

- Para crear una clase de excepción personalizada, escribiremos una clase que hereda de la clase *Exception* incorporada o una de sus subclases, como la clase *ValueError*

- Siempre usaremos un nombre en formato PascalCase acabado en Exception



EXCEPCIONES PERSONALIZADAS

```
class CustomException(Exception):
    pass


try:
    raise CustomException('This is my custom exception')
except CustomException as ex:
    print(ex)
```

EXCEPCIONES PERSONALIZADAS

- Al igual que las clases de excepción estándar, las excepciones personalizadas también son clases. Por lo tanto, podemos agregar funcionalidades a las clases de excepción personalizadas como:

    - atributos y propiedades
    - métodos, por ejemplo, registrar la excepción, formatear la salida, etc.
    - métodos __str__ y __repr__
    - o cualquier otra cosa que podamos hacer con las clases.

- Supongamos que necesitamos desarrollar un programa que convierta una temperatura de Fahrenheit a Celsius.

- Los valores mínimo y máximo de una temperatura en Fahrenheit son 32 y 212. Si los usuarios ingresan un valor que no está en este rango, generaremos una excepción personalizada, por ejemplo, *FahrenheitError*.

EXCEPCIONES PERSONALIZADAS: EJEMPLO

```
class FahrenheitError(Exception):
    min_f = 32
    max_f = 212

    def __init__(self, f, *args):
        super().__init__(args)
        self.f = f

    def __str__(self):
        return f'The {self.f} is not in a valid range'
               f'{self.min_f, self.max_f}'
```

El método __init__ que acepta un valor Fahrenheit (f) y varios argumentos de posición (*args). En el método __init__, llamará al método __init__ de la clase base. Además, asignará el argumento f al atributo de instancia f.

EXCEPCIONES PERSONALIZADAS: EJEMPLO

```
def fahrenheit_to_celsius(f):
    if f < FahrenheitError.min_f or f > \
                FahrenheitError.max_f:
        raise FahrenheitError(f)

    return (f - 32) * 5 / 9
```

EXCEPCIONES PERSONALIZADAS: EJEMPLO

```
if __name__ == '__main__':
    f = input('Enter a temperature in
                    Fahrenheit:')
    try:
        f = float(f)
    except ValueError as ex:
        print(ex)
    else:
        try:
            c = fahrenheit_to_celsius(f)
        except FahrenheitError as ex:
            print(ex)
        else:
            print(f'{f} Fahrenheit = {c:.4f}
                    Celsius')
```

EJERCICIO 8.3